

Modelagem de Dados

Conceitual · Logico · Fisico

Projeto: Projetinho Fellas - Loja de Camisas de Futebol

SGBD: PostgreSQL 14+

ORM: Prisma 5.x

Documento gerado automaticamente a partir de backend/prisma/schema.prisma

Sumario

1. Introducao
2. Identificacao das Entidades
3. Modelo Conceitual (MER - Notacao Chen)
4. Modelo Logico (Esquema Relacional)
5. Modelo Fisico (DDL PostgreSQL)
6. Dicionario de Dados

Apendice A - Schema Prisma

1. Introducao

Este documento apresenta a modelagem de dados completa do sistema **Projetinho Fellas**, um e-commerce de camisas de futebol. A modelagem segue os tres niveis classicos de projeto de banco de dados:

1. Modelo Conceitual - descreve o dominio do problema em alto nivel, de forma independente de tecnologia, atraves do Modelo Entidade-Relacionamento (MER) na notacao Chen.

2. Modelo Logico - traduz o conceitual para o paradigma relacional, identificando tabelas, atributos, chaves primarias, chaves estrangeiras e restricoes.

3. Modelo Fisico - implementacao concreta no SGBD escolhido (PostgreSQL), com tipos de dados nativos, indices e instrucoes DDL.

O sistema possui **6 entidades**: User, Product, ProductSize, Coupon, Order e OrderItem. A autenticao e delegada ao Firebase Auth - apenas o UID do usuario e persistido na tabela User.

2. Identificacao das Entidades

User

Representa o cliente cadastrado na loja. O *id* e o UID gerado pelo Firebase Authentication, garantindo unicidade global. Pode assumir o papel *customer* (cliente comum) ou *admin* (administrador do sistema).

Product

Camisa de futebol disponivel para venda. Mantem catalogo de informacoes do produto (nome, time, descricao, preco, imagens) e o contador *salesCount* usado no ranking do dashboard.

ProductSize

Variacao de tamanho de um Product (P, M, G, GG). Entidade fraca - depende existencialmente de Product. Mantem o controle de estoque por tamanho. Restricao UNIQUE(productId, size).

Coupon

Cupom de desconto. Pode ser de valor fixo (*fixed*) ou percentual (*percent*), com validade definida. O codigo (*code*) e unico e referenciado nos pedidos.

Order

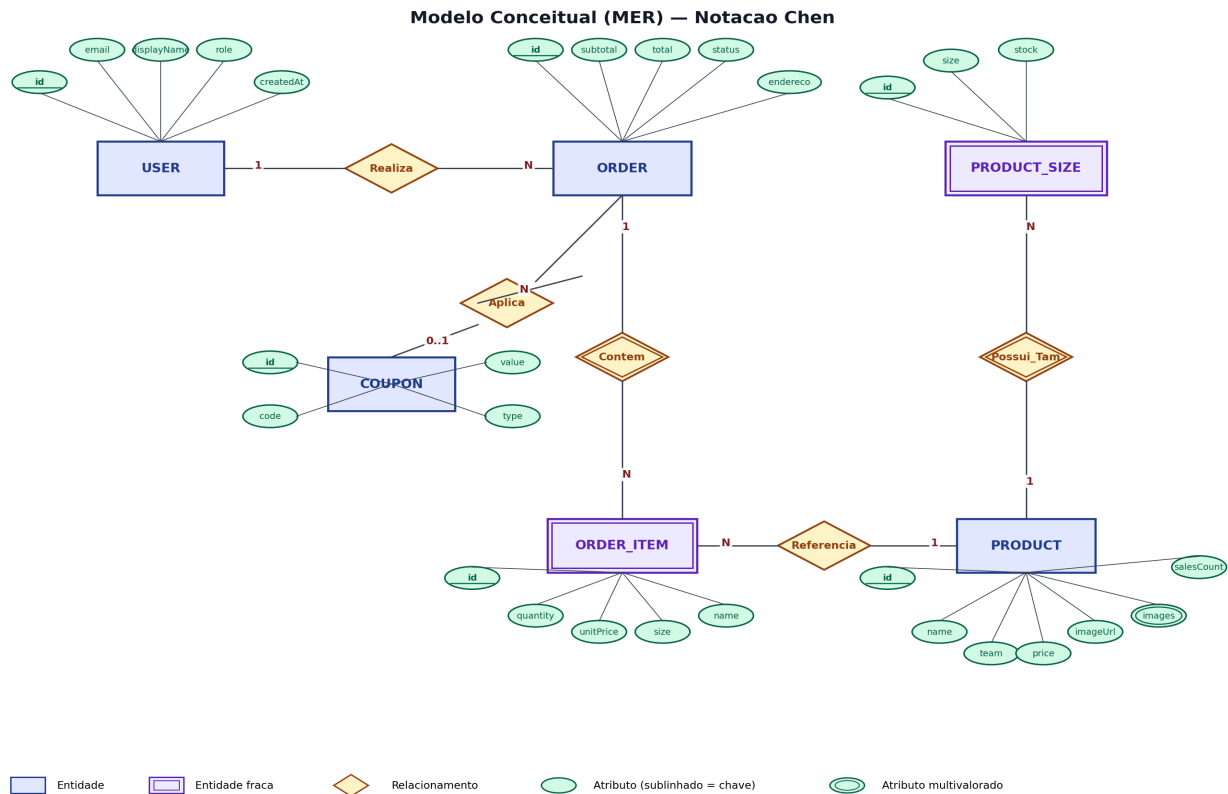
Pedido finalizado pelo cliente. Mantem snapshot do endereco e do historico de status (*statusHistory* em JSON). Total = subtotal - discount + shipping. Status: pendente, pago, enviado, entregue, cancelado.

OrderItem

Item individual dentro de um pedido (entidade fraca de Order). Armazena *snapshots* de nome e preço no momento da compra, garantindo que alterações futuras no produto não corrompam o histórico.

3. Modelo Conceitual (MER - Notacao Chen)

O diagrama abaixo representa as entidades, atributos e relacionamentos do dominio na notacao de Peter Chen. Entidades sao retangulos (duplos para entidades fracas), relacionamentos sao losangos (duplos para identificantes), atributos sao elipses ligadas a entidade dona (sublinhado indica chave; elipse dupla indica multivalorado). As cardinalidades sao anotadas nas linhas.



Relacionamentos e cardinalidades

Relacionamento	Entidades	Cardinalidade	Descricao
Realiza	User - Order	1:N	Um cliente realiza N pedidos; cada pedido pertence a exatamente um cliente.
Aplica_Cupom	Order - Coupon	N:0..1	Um pedido pode aplicar zero ou um cupom; um cupom pode ser usado em N pedidos. Relacionamento implementado via coluna couponCode (string), sem FK forte para preservar o codigo mesmo apos exclusao do cupom.
Contem	Order - OrderItem	1:N (identificante)	Pedido contem N itens. Relacionamento identifiante: OrderItem nao existe sem Order pai. ON DELETE CASCADE.

Relacionamento	Entidades	Cardinalidade	Descricao
Referencia	OrderItem - Product	N:1	Cada item de pedido referencia um produto. Snapshot de nome/preco e mantido no item, permitindo evolucao do catalogo sem alterar pedidos passados. ON DELETE RESTRICT.
Possui_Tamanho	Product - ProductSize	1:N (identificante)	Produto possui N variacoes de tamanho. Relacionamento identificador: tamanho nao existe sem o produto. ON DELETE CASCADE.

4. Modelo Logico (Esquema Relacional)

Traducao do modelo conceitual para o paradigma relacional. Cada entidade vira uma tabela; relacionamentos 1:N viram chaves estrangeiras na entidade fraca; o atributo multivalorado *images* e representado como array textual nativo do PostgreSQL (em SGBDs sem suporte a arrays, seria uma tabela auxiliar PRODUCT_IMAGE).

```
USER (
  id          PK,      -- UID do Firebase Auth
  email       UNIQUE NOT NULL,
  displayName,
  role        NOT NULL DEFAULT 'customer',
  acceptedTerms NOT NULL DEFAULT FALSE,
  createdAt   NOT NULL
)

PRODUCT (
  id          PK,
  name, team, description, price, imageUrl, category NOT NULL,
  images      (array textual, multivalorado),
  salesCount  NOT NULL DEFAULT 0,
  active      NOT NULL DEFAULT TRUE,
  createdAt   NOT NULL
)

PRODUCT_SIZE (
  id          PK,
  productId   FK -> PRODUCT.id  ON DELETE CASCADE,
  size, stock NOT NULL,
  UNIQUE (productId, size)
)

COUPON (
  id          PK,
  code        UNIQUE NOT NULL,
  type        NOT NULL,      -- 'fixed' | 'percent'
  value       NOT NULL,
  validUntil  NOT NULL,
  active      NOT NULL DEFAULT TRUE,
  createdAt   NOT NULL
)

ORDER (
  id          PK,
  userId      FK -> USER.id  ON DELETE RESTRICT,
  userEmail,
  couponCode,      -- referencia fraca a COUPON.code
  subtotal, discount, shipping, total NOT NULL,
  status        NOT NULL DEFAULT 'pendente',
  trackingCode,
  addressFullName, addressStreet, addressNumber,
  addressComplement, addressCity, addressState, addressZip
  -- endereco denormalizado (snapshot)
  statusHistory (JSON),      -- historico de transicoes
  createdAt   NOT NULL
)

ORDER_ITEM (
  id          PK,
  orderId     FK -> ORDER.id    ON DELETE CASCADE,
  productId   FK -> PRODUCT.id  ON DELETE RESTRICT,
  name, size, unitPrice, quantity NOT NULL,
```

```
    imageUrl  
)
```

Normalizacao e denormalizacoes intencionais

Forma normal alcancada: o esquema esta em **3FN** (terceira forma normal). Cada atributo nao-chave depende exclusivamente da chave primaria da sua tabela, e nao ha dependencias transitivas.

Denormalizacoes intencionais (decisoões de projeto):

1. **Endereco dentro de Order** - os 7 campos de endereco (fullName, street, number, complement, city, state, zip) sao denormalizados na tabela ORDER ao inves de criar uma tabela ADDRESS separada. Justificativa: o endereco de entrega de um pedido especifico deve ser imutavel (snapshot), mesmo que o cliente mude seu endereco cadastrado depois.
2. **Snapshot em OrderItem** - os campos *name*, *unitPrice* e *imageUrl* sao duplicados em OrderItem. Justificativa: preserva o estado historico do pedido. Se o preco ou nome do produto mudar, pedidos antigos continuam mostrando os valores reais da compra.
3. **statusHistory como JSON** - ao inves de uma tabela ORDER_STATUS_HISTORY separada, o historico de transicoes e armazenado em coluna *JSONB*. Justificativa: o historico e sempre lido junto com o pedido e nunca consultado isoladamente. JSONB no PostgreSQL mantem indexabilidade quando necessario.
4. **userEmail em Order** - duplica o email do User. Util para listagens administrativas sem necessidade de JOIN, e tambem como snapshot caso o usuario mude o e-mail.
5. **couponCode (string) ao inves de FK** - Order referencia Coupon pelo codigo textual. Permite manter o registro do cupom usado mesmo apos exclusao do cupom.

5. Modelo Fisico (DDL PostgreSQL)

Codigo DDL completo para criacao do schema no PostgreSQL 14+. Os identificadores estao entre aspas duplas para preservar camelCase (padrao gerado pelo Prisma). Tipos nativos do PostgreSQL: TEXT (string), DOUBLE PRECISION (float64), INTEGER, BOOLEAN, TIMESTAMP, JSONB, TEXT[] (array de string).

```
-- =====
-- Projotinho Fellas - DDL PostgreSQL (Modelo Fisico)
-- =====

-- Tabela de usuarios (id e o UID do Firebase Auth)
CREATE TABLE "User" (
  id          TEXT          PRIMARY KEY,
  email       TEXT          NOT NULL UNIQUE,
  "displayName" TEXT,
  role        TEXT          NOT NULL DEFAULT 'customer',
  "acceptedTerms" BOOLEAN   NOT NULL DEFAULT FALSE,
  "createdAt"  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Catalogo de produtos
CREATE TABLE "Product" (
  id          TEXT          PRIMARY KEY,
  name        TEXT          NOT NULL,
  team        TEXT          NOT NULL,
  description  TEXT          NOT NULL DEFAULT '',
  price       DOUBLE PRECISION NOT NULL,
  "imageUrl"  TEXT          NOT NULL,
  images      TEXT[]        NOT NULL DEFAULT '{}',
  category    TEXT          NOT NULL,
  "salesCount" INTEGER      NOT NULL DEFAULT 0,
  active      BOOLEAN       NOT NULL DEFAULT TRUE,
  "createdAt"  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX "Product_category_idx" ON "Product" (category);
CREATE INDEX "Product_active_idx" ON "Product" (active);
CREATE INDEX "Product_salesCount_idx" ON "Product" ("salesCount");

-- Variacoes de tamanho/estoque (entidade fraca de Product)
CREATE TABLE "ProductSize" (
  id          SERIAL PRIMARY KEY,
  "productId" TEXT NOT NULL,
  size        TEXT NOT NULL,
  stock       INTEGER NOT NULL DEFAULT 0,

  CONSTRAINT "ProductSize_productId_size_key" UNIQUE ("productId", size),
  CONSTRAINT "ProductSize_productId_fkey" FOREIGN KEY ("productId")
    REFERENCES "Product"(id) ON DELETE CASCADE ON UPDATE CASCADE
);

-- Cupons de desconto
CREATE TABLE "Coupon" (
  id          TEXT          PRIMARY KEY,
  code        TEXT          NOT NULL UNIQUE,
  type        TEXT          NOT NULL, -- 'fixed' | 'percent'
  value       DOUBLE PRECISION NOT NULL,
  "validUntil"  TIMESTAMP    NOT NULL,
  active      BOOLEAN       NOT NULL DEFAULT TRUE,
  "createdAt"  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```

-- Pedidos (endereço e histórico denormalizados intencionalmente)
CREATE TABLE "Order" (
    id TEXT PRIMARY KEY,
    "userId" TEXT NOT NULL,
    "userEmail" TEXT,
    "couponCode" TEXT,
    subtotal DOUBLE PRECISION NOT NULL,
    discount DOUBLE PRECISION NOT NULL DEFAULT 0,
    shipping DOUBLE PRECISION NOT NULL DEFAULT 0,
    total DOUBLE PRECISION NOT NULL,
    status TEXT NOT NULL DEFAULT 'pendente',
    "trackingCode" TEXT,

    -- Endereço (snapshot do pedido)
    "addressFullName" TEXT NOT NULL,
    "addressStreet" TEXT NOT NULL,
    "addressNumber" TEXT NOT NULL,
    "addressComplement" TEXT,
    "addressCity" TEXT NOT NULL,
    "addressState" TEXT NOT NULL,
    "addressZip" TEXT NOT NULL,

    "statusHistory" JSONB NOT NULL DEFAULT '[]'::jsonb,
    "createdAt" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "Order_userId_fkey" FOREIGN KEY ("userId")
        REFERENCES "User"(id) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE INDEX "Order_userId_idx" ON "Order" ("userId");
CREATE INDEX "Order_status_idx" ON "Order" (status);
CREATE INDEX "Order_createdAt_idx" ON "Order" ("createdAt");

-- Itens de pedido (entidade fraca de Order)
CREATE TABLE "OrderItem" (
    id SERIAL PRIMARY KEY,
    "orderId" TEXT NOT NULL,
    "productId" TEXT NOT NULL,
    name TEXT NOT NULL, -- snapshot
    size TEXT NOT NULL,
    "unitPrice" DOUBLE PRECISION NOT NULL, -- snapshot do preço
    quantity INTEGER NOT NULL,
    "imageUrl" TEXT,

    CONSTRAINT "OrderItem_orderId_fkey" FOREIGN KEY ("orderId")
        REFERENCES "Order"(id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT "OrderItem_productId_fkey" FOREIGN KEY ("productId")
        REFERENCES "Product"(id) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE INDEX "OrderItem_orderId_idx" ON "OrderItem" ("orderId");
CREATE INDEX "OrderItem_productId_idx" ON "OrderItem" ("productId");

```

6. Dicionario de Dados

Detalhamento de cada coluna por tabela, com tipo, restricoes e descricao. PK = Primary Key, FK = Foreign Key, U = UNIQUE, NN = NOT NULL.

User

Coluna	Tipo	Restricoes	Descricao
id	TEXT	PK	UID do Firebase Auth
email	TEXT	U, NN	E-mail unico
displayName	TEXT		Nome de exibicao (opcional)
role	TEXT	NN	'customer' 'admin'
acceptedTerms	BOOLEAN	NN	Aceite dos termos (LGPD)
createdAt	TIMESTAMP	NN	Data de cadastro

Product

Coluna	Tipo	Restricoes	Descricao
id	TEXT	PK	Identificador cuid
name	TEXT	NN	Nome do produto
team	TEXT	NN	Time/selecao
description	TEXT	NN	Descricao (default "")
price	DOUBLE PRECISION	NN	Preco em reais
imageUrl	TEXT	NN	URL da imagem principal
images	TEXT[]	NN	Imagens adicionais (multivalorado)
category	TEXT	NN	Categoria (indexado)
salesCount	INTEGER	NN	Contador de vendas (indexado)
active	BOOLEAN	NN	Ativo para venda (indexado)
createdAt	TIMESTAMP	NN	Data de cadastro

ProductSize

Coluna	Tipo	Restricoes	Descricao
id	SERIAL	PK	Auto-incremento
productId	TEXT	FK->Product, NN	Produto pai (cascade delete)
size	TEXT	NN	P M G GG
stock	INTEGER	NN	Estoque desta variacao
UNIQUE	(productId, size)	U	Sem duplicacao de tamanho por produto

Coupon

Coluna	Tipo	Restricoes	Descricao
id	TEXT	PK	Identificador cuid
code	TEXT	U, NN	Codigo do cupom (uppercase)
type	TEXT	NN	'fixed' 'percent'
value	DOUBLE PRECISION	NN	Valor (R\$ ou %)
validUntil	TIMESTAMP	NN	Data de validade
active	BOOLEAN	NN	Cupom habilitado
createdAt	TIMESTAMP	NN	Data de criacao

Order

Coluna	Tipo	Restricoes	Descricao
id	TEXT	PK	Identificador cuid
userId	TEXT	FK->User, NN	Cliente
userEmail	TEXT		Snapshot do email
couponCode	TEXT		Cupom aplicado (string solta)
subtotal	DOUBLE PRECISION	NN	Soma dos itens
discount	DOUBLE PRECISION	NN	Desconto aplicado
shipping	DOUBLE PRECISION	NN	Frete
total	DOUBLE PRECISION	NN	Total final
status	TEXT	NN	pendente pago enviado entregue cancelado
trackingCode	TEXT		Codigo de rastreio
addressFullName..addressZip	TEXT	NN (maioria)	Endereco snapshot (7 colunas)
statusHistory	JSONB	NN	Array de transicoes
createdAt	TIMESTAMP	NN	Data do pedido

OrderItem

Coluna	Tipo	Restricoes	Descricao
id	SERIAL	PK	Auto-incremento
orderId	TEXT	FK->Order, NN	Pedido pai (cascade)
productId	TEXT	FK->Product, NN	Produto referenciado
name	TEXT	NN	Snapshot do nome
size	TEXT	NN	Tamanho comprado
unitPrice	DOUBLE PRECISION	NN	Snapshot do preco unitario
quantity	INTEGER	NN	Quantidade
imageUrl	TEXT		Snapshot da imagem

Apendice A - Schema Prisma original

Arquivo fonte: backend/prisma/schema.prisma

```
// Schema do banco PostgreSQL – Projétinho Fellas
// docs: https://pris.ly/d/prisma-schema

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id // Firebase Auth UID
  email       String    @unique
  displayName  String?
  role        String    @default("customer") // 'customer' | 'admin'
  acceptedTerms Boolean  @default(false)
  createdAt   DateTime  @default(now())

  orders      Order[]
}

model Product {
  id          String    @id @default(cuid())
  name        String
  team        String
  description  String    @default("")
  price       Float
  imageUrl    String
  images      String[]  @default([])
  category    String
  salesCount  Int        @default(0)
  active      Boolean    @default(true)
  createdAt   DateTime  @default(now())

  sizes       ProductSize[]
  orderItems  OrderItem[]

  @@index([category])
  @@index([active])
  @@index([salesCount])
}

model ProductSize {
  id          Int        @id @default(autoincrement())
  productId   String
  size        String
  stock       Int        @default(0)

  product     Product @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@unique([productId, size])
}

model Coupon {
  id          String    @id @default(cuid())
  code        String    @unique
  type        String    // 'fixed' | 'percent'
  value       Float
}
```

```

    validUntil DateTime
    active      Boolean @default(true)
    createdAt   DateTime @default(now())
}

model Order {
  id          String @id @default(cuid())
  userId      String
  userEmail   String?
  couponCode  String?
  subtotal    Float
  discount    Float @default(0)
  shipping    Float @default(0)
  total       Float
  status      String @default("pendente") // pendente | pago | enviado | entregue | cancelado
  trackingCode String?

  // Endereço (denormalizado – snapshot do pedido)
  addressFullName String
  addressStreet    String
  addressNumber    String
  addressComplement String?
  addressCity      String
  addressState     String
  addressZip       String

  // Histórico de status como JSON: [{ status: string, at: ISOString }, ...]
  statusHistory  Json @default("[]")

  createdAt      DateTime @default(now())

  user           User @relation(fields: [userId], references: [id])
  items          OrderItem[]

  @@index([userId])
  @@index([status])
  @@index([createdAt])
}

model OrderItem {
  id          Int @id @default(autoincrement())
  orderId     String
  productId   String
  name        String // snapshot
  size        String
  unitPrice   Float // snapshot do preço no momento da compra
  quantity    Int
  imageUrl    String?

  order       Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
  product     Product @relation(fields: [productId], references: [id])

  @@index([orderId])
  @@index([productId])
}

```